Insper

Megadados

ACID

Marcel Parolin Jackowski mjack@insper.edu.br
Maciel Vidal macielcv@insper.edu.br
Fábio Ayres fabioja@insper.edu.br

Mais sobre transações

- Existe um conjunto de **propriedades das transações** que tem o propósito de **garantir a validade dos dados armazenados**, mesmo que ocorram
  - falhas de sistema,
  - acessos concorrentes,
  - erros de acesso,
  - etc.

Estas propriedades são indicadas pela sigla **ACID**

ACID

- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability

Atomicidade

Na aula passada vimos como efetuar **transações**, onde

- um conjunto de operações SQL (uma transação)...
- deve ser **concretizada** integralmente através do comando **COMMIT** ...
- ou **cancelada** inteiramente com o comando **ROLLBACK**.

Esta **propriedade**, de garantir que um conjunto de comandos é **executado** ou **rejeitado** como uma única **unidade** é chamada **atomicidade**.

Exercício

Descreva uma situação onde várias operações de banco de dados devem ser executadas **atomicamente**.

Consistência

É a **propriedade** que indica que o **banco de dados** passa de um **estado válido** para **outro estado válido** a cada **transação**.

Consistência

A consistência é estabelecida através de *constraints* e *triggers*. Por exemplo:

- *Constraints* de *foreign key* garantem que um campo de uma tabela (a chave estrangeira) aponta para uma linha válida, e única, de outra tabela.

- Os triggers 'ON DELETE' e 'ON UPDATE' indicam o que fazer quando essas operações ocorrem, para que a base se mantenha consistente.

Exercício

ON DELETE pode ajudar a manter consistência em *deletes* físicos. Como você poderia garantir a consistência do banco de dados na presença de *deletes* lógicos?

Isolamento

É a propriedade relativa à capacidade de executar várias transações concorrentes sem que uma transação interfira diretamente na outra, ou seja, o usuário tem a percepção de que as transações foram executadas sequencialmente.

Exemplo

Suponha que temos dois usuários efetuando as seguintes transações:

Transação 1:

```
START TRANSACTION
SELECT saldo INTO @s FROM contas WHERE id = 1
UPDATE contas SET saldo = saldo - @s/2 WHERE id = 1
UPDATE contas SET saldo = saldo + @s/2 WHERE id = 2
COMMIT
```

Transação 2: o mesmo

```
START TRANSACTION
SELECT saldo INTO @s FROM contas WHERE id = 1
UPDATE contas SET saldo = saldo - @s/2 WHERE id = 1
UPDATE contas SET saldo = saldo + @s/2 WHERE id = 2
COMMIT
```

Vamos pensar um pouco

Se o saldo inicial da conta 1 era 1000 e o saldo inicial da conta 2 era 500, qual será o saldo final de cada conta?

Considere os seguintes cenários:

1. A transação 1 ocorre inteiramente antes da transação 2

2. As operações das duas transações ocorrem alternadamente

Como resolver essa situação?

Parece que o problema é o acesso concorrente às linhas 1 e 2 da tabela *contas*. Você se lembra de algum mecanismo para resolver o problema de acesso concorrente à recursos compartilhados?

Implemente sua solução em pseudo-SQL (tipo, invente uns comandos que façam sentido!)

Uma solução

Transação 1:

```
START TRANSACTION
LOCK contas
SELECT saldo INTO @s FROM contas WHERE id = 1
UPDATE contas SET saldo = saldo - @s/2 WHERE id = 1
UPDATE contas SET saldo = saldo + @s/2 WHERE id = 2
UNLOCK contas
COMMIT
```

Transação 2: o mesmo

```
START TRANSACTION
LOCK contas
SELECT saldo INTO @s FROM contas WHERE id = 1
UPDATE contas SET saldo = saldo - @s/2 WHERE id = 1
UPDATE contas SET saldo = saldo + @s/2 WHERE id = 2
UNLOCK contas
COMMIT
```

Outra solução ...

Transação 1:

```
START TRANSACTION
LOCK contas WHERE id = 1
SELECT saldo INTO @s FROM contas WHERE id = 1
UPDATE contas SET saldo = saldo - @s/2 WHERE id = 1
LOCK contas WHERE id = 2
UPDATE contas SET saldo = saldo + @s/2 WHERE id = 2
UNLOCK contas WHERE id = 2
UNLOCK contas WHERE id = 1
COMMIT
```

Transação 2: o mesmo

```
START TRANSACTION
LOCK contas WHERE id = 1
SELECT saldo INTO @s FROM contas WHERE id = 1
UPDATE contas SET saldo = saldo - @s/2 WHERE id = 1
LOCK contas WHERE id = 2
UPDATE contas SET saldo = saldo + @s/2 WHERE id = 2
UNLOCK contas WHERE id = 2
UNLOCK contas WHERE id = 1
COMMIT
```



Níveis de isolamento

O padrão ANSI/ISO SQL define 4 níveis de **isolamento** de transações:

- `SERIALIZABLE`
- `REPEATABLE READ`
- `READ COMMITED`
- `READ UNCOMMITED`

SERIALIZABLE

Nível mais restritivo, que garante que a execução simultânea de várias transações produza exatamente o mesmo resultado que se fossem executadas de forma estritamente sequencial.

Exercício: O que acontece se todas as transações forem SERIALIZABLE? Liste uma vantagem e um problema.

Este nível é o padrão sugerido pelo comitê ANSI/ISO, mas não é sempre adotado por causa do desempenho. MySQL **NÃO** adota este nível por default.

SERIALIZABLE

#	Trans.	Comando	O que acontece ?	C1 / C2
1	T1	START TRANSACTION		1000 / 500
2	T2	START TRANSACTION		1000 / 500
3	T1	SELECT saldo INTO @s	`@s = 1000`, lock compartilhado na linha 1	1000 / 500
4	T2	SELECT saldo INTO @s	`@s = 1000`, também obtém lock compartilhado (compatível com o de T1)	1000 / 500
5	T1	UPDATE conta 1	Precisa de lock exclusivo, mas T2 segura um compartilhado: **T1 espera**	1000 / 500
6	T2	UPDATE conta 1	Precisa de lock exclusivo, mas T1 segura um compartilhado: **ciclo, deadlock**	1000 / 500
7			O InnoDB detecta na hora e aborta uma transação (erro 1213). Suponha que seja T2: ela sofre `ROLLBACK` e libera seu lock	1000 / 500
8	T1	(UPDATE do passo 5 é liberado)	1000 – 500 = 500	1000 / 500
9	T1	UPDATE conta 2	500 + 500 = 1000	1000 / 500
10	T1	COMMIT	Finaliza	**500 / 1000**

```python
import time
from mysql.connector import errors


ERRO_DEADLOCK = 1213


def transfere_metade(connection):
  cur = connection.cursor()
  cur.execute("SELECT saldo FROM contas WHERE id = %s", (1,))
  s = cur.fetchone()[0]
  cur.execute("UPDATE contas SET saldo = saldo - %s WHERE id = %s", (s / 2, 1))
  cur.execute("UPDATE contas SET saldo = saldo + %s WHERE id = %s", (s / 2, 2))
  connection.commit()


def executa_com_retry(connection, transacao, tentativas=3):
  for tentativa in range(tentativas):
    try:
      transacao(connection) # faz tudo, inclusive o commit
      return
    except errors.Error as e:
      connection.rollback()
      if e.errno == ERRO_DEADLOCK and tentativa < tentativas - 1:
        time.sleep(0.1 * (tentativa + 1)) # pequena espera antes de repetir
      continue
      raise # outro erro, ou acabaram as tentativas


##cur.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
##cur.execute("SELECT ...")
```

REPEATABLE READ

Neste nível,

- se você leu algumas linhas
- e tenta ler elas de novo,
- os mesmos valores serão retornados.

Porém **novas linhas** podem ter sido adicionadas e serão retornadas também na segunda leitura! Esse fenomeno é chamado de *leitura fantasma* (**phantom read**).

Este é o nível *default* de isolamento no engine default do MySQL (InnoDB).

Exemplo REPEATABLE READ

Transação 1	Transação 2
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;	
SELECT * FROM contas WHERE id = 55;	
	UPDATE contas SET saldo = 200 WHERE id = 55; COMMIT;
SELECT * FROM contas WHERE id = 55;	



Uma vez lida, alterações por outras transações não serão visíveis até que esta transação termine!

Exemplo REPEATABLE READ

Transação 1	Transação 2
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;	
SELECT * FROM contas WHERE saldo < 1000;	
	INSERT INTO contas (saldo) VALUES (200); COMMIT;
SELECT * FROM contas WHERE saldo < 1000;	



As linhas da primeira leitura ainda estão aqui, sem modificações! Porém...



... as novas linhas também aparecerão no segundo SELECT! Estas são chamadas de linhas *fantasmas*.

REPEATABLE READ

#	Trans.	Comando	O que acontece ?	C1 / C2
1	T1	START TRANSACTION		1000 / 500
2	T2	START TRANSACTION		1000 / 500
3	T1	SELECT saldo INTO @s	`@s = 1000` (leitura sem trava)	1000 / 500
4	T2	SELECT saldo INTO @s	`@s = 1000` (leitura sem trava)	1000 / 500
5	T1	UPDATE conta 1	saldo = 1000 − 500 = 500, e T1 **trava a linha 1**	1000 / 500
6	T2	UPDATE conta 1	**Bloqueia**: a linha 1 está travada por T1	1000 / 500
7	T1	UPDATE conta 2	conta 2 = 500 + 500 = 1000	1000 / 500
8	T1	COMMIT	Confirma e **libera as travas**	500 / 1000
9	T2	(UPDATE do passo 6 é liberado)	Usa o saldo atual (500) com o `@s` antigo (1000): 500 − 500 = **0**	500 / 1000
10	T2	UPDATE conta 2	1000 + 500 = 1500	500 / 1000
11	T2	COMMIT	Finaliza	**0 / 1500**

READ COMMITTED

Neste nível de isolamento:

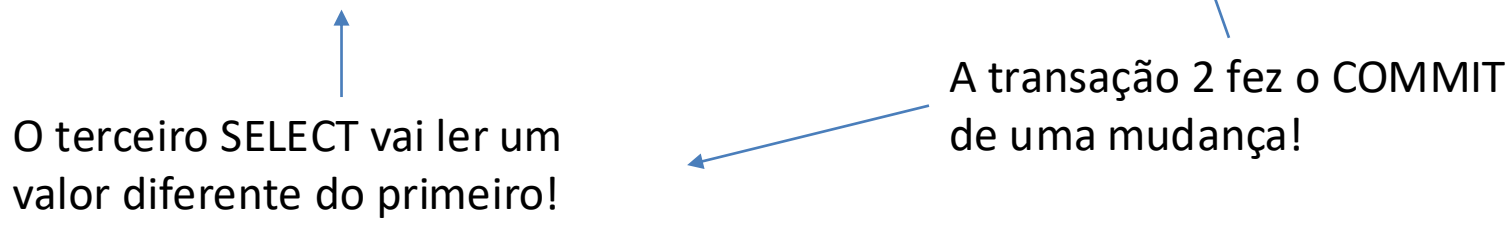
- se uma transação ocorrer em paralelo com a nossa transação ...
- e fizer o COMMIT de seus dados, ...
- então nossa transação pode acabar lendo dados de linha modificados.

Este fenômeno é chamado de "**non-repeatable read**".

Ficará mais claro com um exemplo:

Exemplo READ COMMITTED

Transação 1	Transação 2
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	
SELECT * FROM contas WHERE id = 55;	
	UPDATE contas SET saldo = 200 WHERE id = 55;
SELECT * FROM contas WHERE id = 55;	
	COMMIT;
SELECT * FROM contas WHERE id = 55;	



READ UNCOMMITTED

Neste nível de isolamento um dado modificado de uma transação não-finalizada será acessível pela nossa transação (**dirty read**). É o nível mais perigoso, e raramente utilizado.

Exemplo READ UNCOMMITTED

Transação 1	Transação 2
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITED;	
SELECT * FROM contas WHERE id = 55;	
	UPDATE contas SET saldo = 200 WHERE id = 55;
SELECT * FROM contas WHERE id = 55;	
	ROLLBACK;

A transação 2 ainda não fez o COMMIT de uma mudança!

O segundo SELECT vai ler um valor diferente do primeiro!

Ops, ninguém deveria ter lido o valor alterado! A transação 1 leu lixo!

Comparação

Nível	Dirty reads?	Non-repeatable reads?	Phantom reads?
SERIALIZABLE	Não	Não	Não
REPEATABLE READ	Não	Não	Pode ocorrer
READ COMMITED	Não	Pode ocorrer	Pode ocorrer
READ UNCOMMITED	Pode ocorrer	Pode ocorrer	Pode ocorrer

Durabilidade

É a **propriedade** que diz que quando uma transação é **confirmada** (COMMIT), ela **permanecerá gravada** mesmo que a energia acabe ou o sistema trave.

Próxima aula

Capítulo 15

- Stored procedures
- Constraints e triggers

Insper

www.insper.edu.br